OBJECT ORIENTED DESIGN

QUIZ 1 – SKPL DOCUMENT

KHEN MUHAMMAD CAHYO

244107023002

TI-2I

STATE POLYTECHNIC OF MALANG

INFORMATION TECHNOLOGY DEPARTMENT

SOFTWARE SYSTEM REQUIREMENTS

STUDENT UKM SELECTION DETERMINATION SYSTEM

Version 1.0.0

Prepared by - Khen Muhammad Cahyo

March, 13 2025

1. **Functional Requirement**

1.1. Use Case Diagram

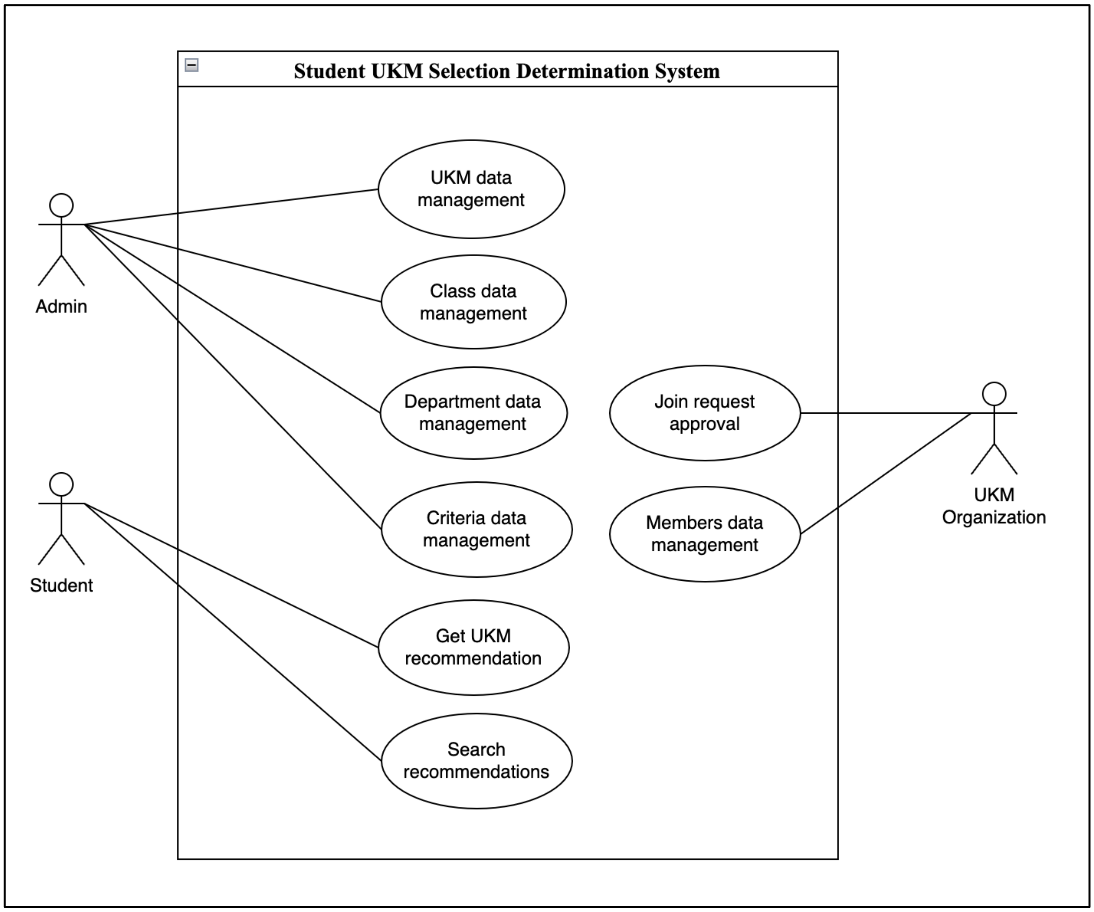
- **Cases**
  - **System Admin** can manage master data such as criteria, UKM, classes, and departments as the main data used in the selection system.
  - **Students** can request UKM recommendations by inputting name, email, nim, class, and department data and selecting the desired criteria.
  - **Students** also can search their previous recommendations data by search the data from input in recommendations data collections.
  - **UKM Organization** can approve join request from students on their UKM.
  - **UKM Organization** can manage members data on their UKM.
- **Actors & Goals**
  - **Actors**

    Based on previous explained use cases, there is 3 actors that will be used on this systems.

    1. Admin
    2. Student
    3. UKM Organization
  - **Goals**

    The purpose of this system is to provide recommendations to students when students are confused about choosing which UKM is in accordance with their abilities and other factors.

- **Diagram**



1.2. Use Case Description

- **Get UKM Recommendation**

Use Case Name: Get UKM Recommendation	**ID**: UCD-001	**Importance Level**: High
Primary Actor: Student	**Use Case Type**: Detail, Overview	
Stakeholder and Interest: Student – Want to get UKM recommendation for their personals.		
Brief Description: This use case described how students can get UKM recommendation on this app.		
Trigger: Student click Get Recommendation button on get recommendation page after filled form inputs.		
Type: Temporal Trigger		

Relationship:
Normal Flow Event: 1. Student open the app on browser. 2. Student fill form inputs (name, email, nim, class, department, and their criterias) and submit the data. 3. System will calculates the criterias from student. 4. After that, system will give student an recommendations based on the calculating results. 5. Student can select one or more UKM from recommendations results and register on UKM that they selected.
Sub Flows: 1. System admin created master data for get UKM recommendations needs (classes, departments, ukm, and criterias). 2. System admin ensure that this data can accessed by students in get UKM recommendations feature.
Alternate/Exceptional Flow: If student don't get an recommendations from system, they can submit request again with different criterias.

- **Search Recommendations**

Use Case Name: Search Recommendation	**ID**: UCD-002	**Importance Level**: High
Primary Actor: Student	**Use Case Type**: Detail, Essential	
Stakeholder and Interest: Student – Search recommendations and join requests status for their personals.		
Brief Description: This use case described how student can check their recommendations and UKM join requests approval status.		
Trigger: Search input on recommendations table.		
Type: Temporal Trigger		

Relationship:
Normal Flow Event: 1. Student open the app on browser. 2. On landing page, students will see table recommendations results from all students that has request on this app. 3. In search input, students can search their recommendations data based on their name. 4. After that, student can see their recommendations and their UKM join requests approval status.
Sub Flows:
Alternate/Exceptional Flow:

- **Join Request Approval**

Use Case Name: Join Request Approval	**ID**: UCD-003	**Importance Level**: High
Primary Actor: UKM Organization	**Use Case Type**: Detail, Real	
Stakeholder and Interest: UKM Organization – Approve UKM join requests from students.		
Brief Description: This use case described how student can check their recommendations and UKM join requests approval status.		
Trigger: Approve or Reject button.		
Type: Temporal Trigger		
Relationship:		
Normal Flow Event: 1. UKM Organization open members candidate page that showing all of members candidate data in table. 2. UKM Organization choose one of the members candidate.		

3. UKM Organization click on Actions button and it will show 2 options (Approve and Reject). 4. UKM Organization give an approval answer for candidates based on 2 options before.
Sub Flows: 1. System admin registered one account for one organization • System admin give the registered account to UKM Organization leader 2. UKM Organization login on the app for give approval to members candidate with input username and password. 3. After UKM Organization give approval results, Students who are given approval status will receive notification on their email.
Alternate/Exceptional Flow: 1. If UKM Organization leader forget they password, they can request reset password by email.

1.3. Class Diagram

- **Identify Entities**

  Here are each entities that will used on this case.

  - Department
  - Class
  - Student
  - UKM
  - StudentUKM
  - Criteria
  - Assessment
  - Recommendation
  - UKMCriteria

- **Entity Attributes & Methods**

  Here are the attributes and methods that is owned by each entities:

  - **Department**
    - #id: string
    - #name: string
    - #email: string
    - #nim: string
    - #class: Class
    - +createStudent(name, nim, class)
    - +getStudentById(id)
    - +getUkmRecommendation()
  - **Class**
    - #id: string
    - #name: string
    - #department: Department
    - +createClass(name, department)
    - +getAllClass()
    - +getClassById(id)
  - **Student**
    - #id: string
    - #name: string
    - #email: string
    - #nim: string
    - #class: Class
    - +createStudent(name, nim, class)
    - +getStudentById(id)
    - +getUkmRecommendation()

- **UKM**
  - #id: string
  - #name: string
  - #description: string
  - #leader: Student
  - +createUkm(name, description, leader)
  - +getAllUkm()
  - +getUkmById(id)
- **StudentUKM**
  - #id: string
  - #student: Student
  - #ukm: UKM
  - +getStudentUkm(student)
- **Criteria**
  - #id: string
  - #name: string
  - #weight: float
  - +createCriteria(name, weight)
  - +getAllCriteria()
  - +getCriteriaById(id)
- **Assessment**
  - #id: string
  - #name: string
  - #score: float
  - +getStudentAssessments(student)
  - +getStudentRecommendation(student)
- **Recommendation**
  - #id: string
  - #ukm: UKM

    - **UKMCriteria**
        - #ukm: UKM
        - #criteria: Criteria

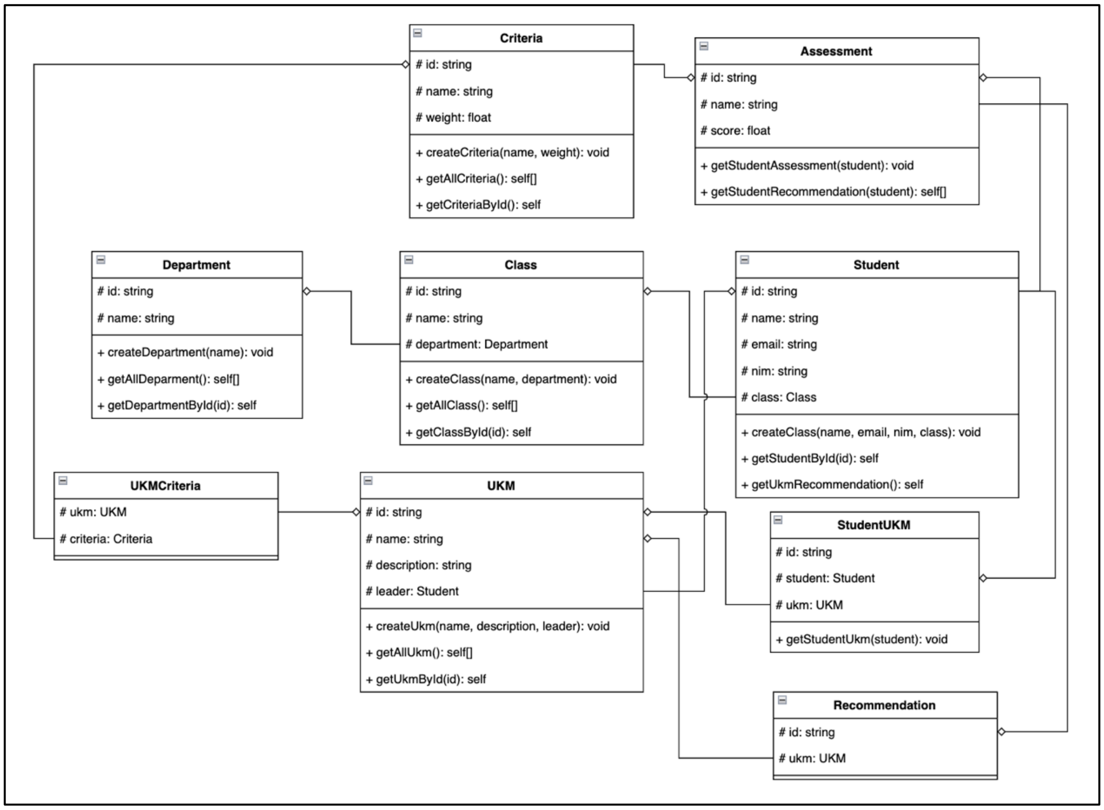
- **Relationship Between Entities**

  Here are explanation about relationship between entities that will used on this case.

    - **Department – Class**
        - **Relations**: Composition (1-to-Many)
        - **Explanations**: 1 Department has many Class, but Class can’t exist without Department.
    - **Class – Student**
        - **Relations**: Association (1-to-Many)
        - **Explanations**: 1 Class can accommodate many Student, and Student must registered on 1 Class.
    - **Student – StudentUKM**
        - **Relations**: Aggregation (1-to-Many)
        - **Explanations**: 1 Student can follows many UKM, and StudentUKM only connector table.
    - **Student – Assessment**
        - **Relations:** Composition (1-to-Many)
        - **Explanations:** 1 Student has many Assessment, but Assessment can’t exist without Student.
    - **StudentUKM – UKM**
        - **Relations:** Association (Many-to-Many)
        - **Explanations:** Many Student can joins with many UKM.
    - **Assessment – Criteria**
        - **Relations:** Association (1-to-1)
        - **Explanations:** Each Assessment assesses 1 specific criteria, allows to tracking per-criteria value.

- **UKM – UKMCriteria**
  - **Relations:** Composition (1-to-Many)
  - **Explanations:** 1 UKM has many Criteria with different weight, determine priority for UKM assessments.
- **UKMCriteria – Criteria**
  - **Relations:** Association (Many-to-Many)
  - **Explanations:** Store criteria-specific weights for each UKM, allows for weight variation between UKMs.
- **Recommendation – Assessment**
  - **Relations:** Dependency (1-to-1)
  - **Explanations:** Recomendation generated based on Assessment calculation, and 1
- **Recommendation – UKM**
  - **Relations:** Dependency (1-to-1)
  - **Explanations:** Every Recommendation leading to specific UKM, demonstrate Student's fit with UKMs.
- **UKM – Student (Leader)**
  - **Relations:** Association (1-to-1)
  - **Explanations:** Every UKM led by 1 Student, and Leader is a registered UKM members.

- **Diagram**



Criteria
id: string
name: string
weight: float
+ createCriteria(name, weight): void
+ getAllCriteria(): self[]
+ getCriteriaById(): self
Assessment
id: string
name: string
score: float
+ getStudentAssessment(student): void
+ getStudentRecommendation(student): self[]
Department
id: string
name: string
+ createDepartment(name): void
+ getAllDeparment(): self[]
+ getDepartmentById(id): self
Class
id: string
name: string
department: Department
+ createClass(name, department): void
+ getAllClass(): self[]
+ getClassById(id): self
Student
id: string
name: string
email: string
nim: string
class: Class
+ createClass(name, email, nim, class): void
+ getStudentById(id): self
+ getUkmRecommendation(): self
UKMCriteria
ukm: UKM
criteria: Criteria
UKM
id: string
name: string
description: string
leader: Student
+ createUkm(name, description, leader): void
+ getAllUkm(): self[]
+ getUkmById(id): self
StudentUKM
id: string
student: Student
ukm: UKM
+ getStudentUkm(student): void
Recommendation
id: string
ukm: UKM